Filter Module — Design & Architecture

Version: 1.0
Date: April 15, 2026
Author: Architecture Team
Parent Doc: banking-application-architecture.md

Table of Contents

1. Problem Statement
2. Filter Dimensions & Taxonomy
3. Architecture Overview
4. Frontend Filter Design
5. Backend Filter Pipeline — Specification Pattern
6. Filter vs Search — Routing Decision
7. PostgreSQL Query Optimization for Filters
8. Dynamic Filter Options (Facets)
9. URL State Synchronization
10. Pagination & Sorting Strategy
11. Performance Engineering
12. Edge Cases & Error Handling
13. GraphQL vs REST for Filters — Decision Rationale

---

1. Problem Statement

A banking customer with multiple accounts generates thousands of transactions over time across different payment types (UPI, NEFT, RTGS, EMI, Bill Pay), amounts, and statuses. The transaction history screen must allow users to slice through this data efficiently using multiple filter criteria simultaneously — without losing context, with instant feedback, and with shareable/bookmarkable URLs.

Key challenges:

- Filters are combinatorial: 6 filter dimensions with multiple values each can produce thousands of filter combinations.
- Filters must be composable: applying category=UPI should not break the date range filter already active.
- Filter state must survive page refresh, browser back/forward, and sharing via URL.
- Backend must translate arbitrary filter combinations into efficient SQL without N+1 or full-scan scenarios.

---

2. Filter Dimensions & Taxonomy

2.1 Complete Filter Specification

```
┌──────────────────────────────────────────────────────────────────────┐
│                    MULTI-LEVEL FILTER PANEL                          │
│                                                                      │
│  LEVEL 1: Account         LEVEL 2: Date Range       LEVEL 3: Category  │
│  ┌──────────────────┐     ┌──────────────────┐      ┌────────────────┐ │
│  │ Multi-select     │     │ Date picker      │      │ Checkbox group │ │
│  │                  │     │                  │      │                │ │
│  │ ☑ Savings (XXX1) │     │ From: [________] │      │ ☑ All          │ │
│  │ ☑ Current (XXX2) │     │ To:   [________] │      │ ☐ UPI          │ │
│  │ ☐ Loan (XXX3)    │     │                  │      │ ☐ NEFT         │ │
│  │                  │     │ Quick picks:     │      │ ☐ RTGS         │ │
│  │ [Select All]     │     │ [Today] [7d]     │      │ ☐ IMPS         │ │
│  │                  │     │ [30d] [90d]      │      │ ☐ Bill Pay     │ │
│  │                  │     │ [This month]     │      │ ☐ EMI          │ │
│  │                  │     │ [Custom range]   │      │ ☐ ATM          │ │
│  └──────────────────┘     └──────────────────┘      └────────────────┘ │
│                                                                      │
│  LEVEL 4: Amount Range    LEVEL 5: Status           LEVEL 6: Type      │
│  ┌──────────────────┐     ┌──────────────────┐      ┌────────────────┐ │
│  │ Dual range slider │     │ Radio group      │      │ Radio group    │ │
│  │                  │     │                  │      │                │ │
│  │ Min: [₹_______]  │     │ ○ All            │      │ ○ All          │ │
│  │ Max: [₹_______]  │     │ ○ Success        │      │ ○ Credit       │ │
│  │                  │     │ ○ Pending        │      │ ○ Debit        │ │
│  │ Presets:         │     │ ○ Failed         │      │                │ │
│  │ [< ₹1K]          │     │                  │      │                │ │
│  │ [₹1K - ₹10K]     │     │                  │      │                │ │
│  │ [₹10K - ₹1L]     │     │                  │      │                │ │
│  │ [> ₹1L]          │     │                  │      │                │ │
│  └──────────────────┘     └──────────────────┘      └────────────────┘ │
│                                                                      │
│  ┌──────────────────────────────────────────────────────────────────┐│
│  │ ACTIVE FILTERS:                                                  ||
│  │ [Savings ×] [Last 30 days ×] [UPI ×] [< ₹10K ×]    [Clear All]   ||
│  └──────────────────────────────────────────────────────────────────┘│
│                                                                      │
│  Showing 47 of 1,234 transactions                    [Apply Filters] │
└──────────────────────────────────────────────────────────────────────┘
```

2.2 Filter Dimension Details

Dimension	Type	Default	Multi-Select?	Backend Column	Index Used
Account	Multi-select dropdown	All user accounts	Yes	`account_id`	Composite: `(account_id, created_at DESC)`
Date Range	Date picker + presets	Last 30 days	No (single range)	`created_at`	Partition pruning + composite index
Category	Checkbox group	All	Yes	`category`	`idx_txn_category`
Amount Range	Dual slider / input	No limit	No (single range)	`amount`	`idx_txn_amount`
Status	Radio buttons	All	No (single value)	`status`	Partial index: `WHERE status ≠ 'SUCCESS'`
Type	Radio buttons	All	No (single value)	`type`	`idx_txn_type`

3. Architecture Overview

```
                        FILTER ARCHITECTURE
                        ___________________


┌──────────────────────────────────────────────────────────────────┐
│                        REACT FRONTEND                            │
│                                                                  │
│  ┌──────────────┐   ┌──────────────────┐   ┌────────────────────┐  │
│  │ FilterPanel  │   │ useTransaction   │   │ URL State Manager  │  │
│  │ Component    │──►│ Filters (hook)   │──►│ (sync to ?params)  │  │
│  │              │   │                  │   │                    │  │
│  │ • Renders    │   │ • Reducer-based  │   │ • Encodes filters  │  │
│  │   all filter │   │   state mgmt     │   │   to URL params    │  │
│  │   controls   │   │ • Debounces (300ms)│ │ • Decodes on load  │  │
│  │ • Emits      │   │ • Triggers API   │   │ • Browser history  │  │
│  │   changes    │   │   via RTK Query  │   │   integration      │  │
│  └──────────────┘   └──────────────────┘   └────────────────────┘  │
│        │                  │                                      │
│        ▼                  ▼                                      │
│  ┌──────────────┐   ┌──────────────────┐                         │
│  │ FilterChips  │   │ TransactionList  │                         │
│  │ (active tags) │  │ (paginated)      │                         │
│  └──────────────┘   └──────────────────┘                         │
└──────────────────────────────────────────────────────────────────┘
                              │
                              │  GET /api/v1/transactions?accountIds= … &from= … &categories= …
                              ▼
┌──────────────────────────────────────────────────────────────────┐
│                          BFF-Web                                 │
│  • Validates filter params (sanitize, type-check)                │
│  • Injects userId from JWT (server-side, not from client)        │
│  • Forwards to Transaction Service via gRPC                      │
└──────────────────────────────────────────────────────────────────┘
                              │
                              ▼
┌──────────────────────────────────────────────────────────────────┐
│                     TRANSACTION SERVICE                          │
│                                                                  │
│  ┌──────────────────────────────────────────────────────────┐    │
│  │              Specification Builder                        │    │
│  │                                                           │    │
│  │  TransactionFilterDTO ──► Specification<Transaction>      │    │
│  │                                                           │    │
│  │  Each filter dimension → one Specification                │    │
│  │  All specs ANDed together via Specification.and()         │    │
│  │  Empty/null filters → no-op (spec omitted)               │    │
│  └──────────────────────────────────────────────────────────┘    │
│        │                                                         │
│        ▼                                                         │
│  ┌──────────────────────────────────────────────────────────┐    │
│  │           JPA + PostgreSQL (via JpaSpecificationExecutor)  │    │
│  │                                                           │    │
│  │  SELECT * FROM transactions                               │    │
│  │  WHERE account_id IN ( … )                                │    │
│  │    AND created_at BETWEEN … AND …                         │    │
│  │    AND category IN ( … )                                  │    │
│  │    AND amount BETWEEN … AND …                             │    │
│  │    AND status = ' … '                                     │    │
│  │    AND type = ' … '                                       │    │
│  │  ORDER BY created_at DESC                                 │    │
│  │  LIMIT 20 OFFSET 0                                       │    │
│  └──────────────────────────────────────────────────────────┘    │
└──────────────────────────────────────────────────────────────────┘
```

4. Frontend Filter Design

4.1 Filter State Model

```
interface TransactionFilters {
  accountIds:  string[];                              // multi-select
  dateRange:   { from: string; to: string } | null;   // ISO date strings
  categories:  TransactionCategory[];                  // multi-select enum
  amountRange: { min: number; max: number } | null;   // nullable = no limit
  status:      'ALL' | 'SUCCESS' | 'PENDING' | 'FAILED';
  type:        'ALL' | 'CREDIT' | 'DEBIT';
  page:        number;
  pageSize:    number;                                 // 20 | 50 | 100
  sortBy:      'createdAt' | 'amount';
  sortDir:     'asc' | 'desc';
}

type TransactionCategory = 'UPI' | 'NEFT' | 'RTGS' | 'IMPS' | 'BILL_PAY' | 'EMI' | 'ATM';

const DEFAULT_FILTERS: TransactionFilters = {
  accountIds:  [],            // empty = all user accounts
  dateRange:   { from: thirtyDaysAgo(), to: today() },
  categories:  [],            // empty = all categories
  amountRange: null,
  status:      'ALL',
  type:        'ALL',
  page:        0,
  pageSize:    20,
  sortBy:      'createdAt',
  sortDir:     'desc'
};
```

4.2 Filter Reducer

```
type FilterAction =
  | { type: 'SET_ACCOUNTS';     payload: string[] }
  | { type: 'SET_DATE_RANGE';   payload: { from: string; to: string } | null }
  | { type: 'TOGGLE_CATEGORY';  payload: TransactionCategory }
  | { type: 'SET_AMOUNT_RANGE'; payload: { min: number; max: number } | null }
  | { type: 'SET_STATUS';       payload: TransactionFilters['status'] }
  | { type: 'SET_TYPE';         payload: TransactionFilters['type'] }
  | { type: 'SET_PAGE';         payload: number }
  | { type: 'SET_PAGE_SIZE';    payload: number }
  | { type: 'SET_SORT';         payload: { sortBy: string; sortDir: 'asc' | 'desc' } }
  | { type: 'RESET_ALL' }
  | { type: 'HYDRATE_FROM_URL'; payload: Partial<TransactionFilters> };

function filterReducer(state: TransactionFilters, action: FilterAction): TransactionFilters {
  switch (action.type) {
    case 'SET_ACCOUNTS':
      return { ...state, accountIds: action.payload, page: 0 };

    case 'SET_DATE_RANGE':
      return { ...state, dateRange: action.payload, page: 0 };

    case 'TOGGLE_CATEGORY': {
      const cat = action.payload;
      const current = state.categories;
      const updated = current.includes(cat)
        ? current.filter(c => c !== cat)
        : [...current, cat];
      return { ...state, categories: updated, page: 0 };
    }

    case 'SET_AMOUNT_RANGE':
      return { ...state, amountRange: action.payload, page: 0 };

    case 'SET_STATUS':
      return { ...state, status: action.payload, page: 0 };

    case 'SET_TYPE':
      return { ...state, type: action.payload, page: 0 };

    case 'SET_PAGE':
      return { ...state, page: action.payload };

    case 'RESET_ALL':
      return DEFAULT_FILTERS;

    case 'HYDRATE_FROM_URL':
      return { ...DEFAULT_FILTERS, ...action.payload };

    default:
      return state;
  }
}
```

Page reset on filter change: Every filter mutation (except pagination) resets `page` to 0. If a user is on page 5, changing category=UPI brings them back to page 1 of the new result set.

4.3 Debouncing Strategy

```
function useTransactionFilters() {
  const [filters, dispatch] = useReducer(filterReducer, DEFAULT_FILTERS);
  const debouncedFilters = useDebounce(filters, 300);

  const { data, isLoading, isFetching } = useQuery(
    ['transactions', debouncedFilters],
    () => transactionApi.list(debouncedFilters),
    {
      keepPreviousData: true,   // show stale results while new ones load
      staleTime: 30_000,        // refetch only after 30s
      refetchOnWindowFocus: false
    }
  );

  useSyncFiltersToURL(filters, dispatch);

  return { filters, dispatch, data, isLoading, isFetching };
}
```

- **300ms debounce:** Prevents API call on every keystroke (amount input) or rapid checkbox clicks.
- **`keepPreviousData: true`:** While fetching new results, the UI continues showing the previous results (no blank flicker). A subtle loading indicator shows new data is being fetched.
- **`staleTime: 30_000`:** If the user navigates away and comes back within 30 seconds, the cached data is served immediately.

4.4 Filter Chips Component

Active filters are displayed as removable chips above the transaction list:

```
function FilterChips({ filters, dispatch }: FilterChipsProps) {
  const chips: Chip[] = [];

  if (filters.accountIds.length > 0) {
    filters.accountIds.forEach(id => {
      chips.push({
        label: accountName(id),
        onRemove: () => dispatch({
          type: 'SET_ACCOUNTS',
          payload: filters.accountIds.filter(a => a !== id)
        })
      });
    });
  }

  if (filters.dateRange) {
    chips.push({
      label: formatDateRange(filters.dateRange),
      onRemove: () => dispatch({ type: 'SET_DATE_RANGE', payload: null })
    });
  }

  // ... chips for categories, amount, status, type

  return (
    <div className="flex flex-wrap gap-2">
      {chips.map(chip => (
        <Badge key={chip.label} variant="outline" className="gap-1">
          {chip.label}
          <button onClick={chip.onRemove} aria-label={`Remove ${chip.label}`}>×</button>
        </Badge>
      ))}
      {chips.length > 1 && (
        <button onClick={() => dispatch({ type: 'RESET_ALL' })}>Clear All</button>
      )}
    </div>
  );
}
```

---

5. Backend Filter Pipeline — Specification Pattern

5.1 Why JPA Specifications?

Approach	Problem
Raw SQL string concatenation	SQL injection risk. Hard to maintain. No type safety.
Multiple repository methods	`findByAccountIdAndStatusAndCategory( ... )` — combinatorial explosion. 2^6 = 64 methods for 6 filters.
@Query with conditional JPQL	Messy with many optional params. Hibernate generates suboptimal SQL for null checks.
Criteria API directly	Verbose, error-prone, hard to read.
Specification Pattern (chosen)	Each filter is a composable, testable, reusable `Specification<T>`. They chain via `.and()`. Null/empty filters simply don't add a spec. Clean, extensible, type-safe.

5.2 Specification Implementations

```java
public final class TransactionSpecs {

    public static Specification<Transaction> accountIdIn(List<String> accountIds) {
        return (root, query, cb) -> root.get("accountId").in(accountIds);
    }

    public static Specification<Transaction> dateBetween(Instant from, Instant to) {
        return (root, query, cb) -> cb.between(root.get("createdAt"), from, to);
    }

    public static Specification<Transaction> categoryIn(List<String> categories) {
        return (root, query, cb) -> root.get("category").in(categories);
    }

    public static Specification<Transaction> amountBetween(BigDecimal min, BigDecimal max) {
        return (root, query, cb) -> {
            if (min != null && max != null) return cb.between(root.get("amount"), min, max);
            if (min != null) return cb.greaterThanOrEqualTo(root.get("amount"), min);
            if (max != null) return cb.lessThanOrEqualTo(root.get("amount"), max);
            return cb.conjunction();
        };
    }

    public static Specification<Transaction> statusEquals(String status) {
        return (root, query, cb) -> cb.equal(root.get("status"), status);
    }

    public static Specification<Transaction> typeEquals(String type) {
        return (root, query, cb) -> cb.equal(root.get("type"), type);
    }
}
```

5.3 Specification Builder — Composing Filters

```java
public class TransactionSpecBuilder {

    public static Specification<Transaction> build(TransactionFilterDTO f, String userId) {
        // Mandatory: scope to user's accounts only
        Specification<Transaction> spec = Specification.where(
            accountIdIn(getUserAccountIds(userId))
        );

        // Optional filters - only applied if present
        if (isNotEmpty(f.getAccountIds())) {
            // Intersect user's accounts with requested accounts (prevents IDOR)
            List<String> allowed = f.getAccountIds().stream()
                .filter(getUserAccountIds(userId)::contains)
                .toList();
            spec = spec.and(accountIdIn(allowed));
        }

        if (f.getFrom() != null && f.getTo() != null)
            spec = spec.and(dateBetween(f.getFrom(), f.getTo()));

        if (isNotEmpty(f.getCategories()))
            spec = spec.and(categoryIn(f.getCategories()));

        if (f.getMinAmount() != null || f.getMaxAmount() != null)
            spec = spec.and(amountBetween(f.getMinAmount(), f.getMaxAmount()));

        if (f.getStatus() != null && !"ALL".equals(f.getStatus()))
            spec = spec.and(statusEquals(f.getStatus()));

        if (f.getType() != null && !"ALL".equals(f.getType()))
            spec = spec.and(typeEquals(f.getType()));

        return spec;
    }
}
```

5.4 Generated SQL Example

For filters: `accountIds=ACC-001&from=2026-01-01&to=2026-03-31&categories=UPI,NEFT&status=SUCCESS&sortBy=createdAt&sortDir=desc&page=0&size=20`

```sql
SELECT t.id, t.account_id, t.amount, t.type, t.category, t.status,
       t.description, t.reference_id, t.created_at, t.updated_at
FROM transactions t
WHERE t.account_id IN ('ACC-001')
  AND t.created_at ≥ '2026-01-01 00:00:00+00'
  AND t.created_at ≤ '2026-03-31 23:59:59+00'
  AND t.category IN ('UPI', 'NEFT')
  AND t.status = 'SUCCESS'
ORDER BY t.created_at DESC
LIMIT 20 OFFSET 0;
```

PostgreSQL query plan leverages:

1. **Partition pruning:** Only scans `transactions_2026_01`, `transactions_2026_02`, `transactions_2026_03`.
2. **Index scan:** Uses composite index `(account_id, created_at DESC)`.
3. **Filter on category/status:** Applied as index filter conditions.

---

6. Filter vs Search — Routing Decision

Filters and search are complementary but hit different backends:

```
┌──────────────────────────────────────────────────────────────┐
│                    REQUEST ROUTING                            │
│                                                               │
│  Frontend sends: { q: "swiggy", categories: ["UPI"], ... }    │
│                                                               │
│  ┌──────────────────────────────────────────────────────────┐ │
│  │ if (q is present && q.length ≥ 2)                        │ │
│  │     → ELASTICSEARCH                                      │ │
│  │       text query in ES "must" clause                     │ │
│  │       structured filters in ES "filter" clause           │ │
│  │       (both executed in single ES query)                 │ │
│  │                                                          │ │
│  │ else                                                     │ │
│  │     → POSTGRESQL                                         │ │
│  │       JPA Specification pipeline                         │ │
│  │       (faster for structured filters, avoids ES overhead) │ │
│  └──────────────────────────────────────────────────────────┘ │
│                                                               │
│  Key: filters always work regardless of which engine handles it │
└──────────────────────────────────────────────────────────────┘
```

When ES handles the query, structured filters are applied as `bool.filter` clauses (not `bool.must`). This means they don't affect relevance scoring but do restrict results. ES caches filter clauses automatically for repeated queries.

---

7. PostgreSQL Query Optimization for Filters

7.1 Index Strategy

```sql
-- Primary composite index: covers most common filter pattern
-- (account lookup + date ordering, which is the default view)
CREATE INDEX idx_txn_account_date ON transactions (account_id, created_at DESC);

-- Category filter index
CREATE INDEX idx_txn_category ON transactions (category);

-- Amount range index
CREATE INDEX idx_txn_amount ON transactions (amount);

-- Partial index for non-success statuses (most txns are SUCCESS, so this is small)
CREATE INDEX idx_txn_status_partial ON transactions (status) WHERE status ≠ 'SUCCESS';

-- Full-text search fallback (used when ES is down)
CREATE INDEX idx_txn_description_fts ON transactions USING gin(to_tsvector('english', description));
```

7.2 Partition Pruning

The `transactions` table is range-partitioned by `created_at` (monthly). When a date range filter is applied, PostgreSQL's query planner automatically prunes partitions:

```
Date filter: from=2026-02-01, to=2026-03-15

Partitions scanned:
  ✓ transactions_2026_02
  ✓ transactions_2026_03
  ✗ transactions_2026_01  (pruned)
  ✗ transactions_2026_04  (pruned)
  ✗ ... all other months  (pruned)
```

This is why the date range filter is the single most important performance lever — it determines how many partitions are touched.

7.3 EXPLAIN ANALYZE Output

```
Sort  (cost=15234.56..15234.61 rows=20 width=256) (actual time=12.345..12.350 rows=20 loops=1)
  Sort Key: created_at DESC
  Sort Method: top-N heapsort  Memory: 28kB
  →  Append  (cost=0.43..15200.00 rows=47 width=256) (actual time=0.05..11.89 rows=47 loops=1)
        →  Index Scan using transactions_2026_02_account_date_idx on transactions_2026_02
              Index Cond: (account_id = 'ACC-001' AND created_at ≥ '2026-02-01' AND created_at ≤ '2026-03-15')
              Filter: (category = ANY ('{UPI,NEFT}') AND status = 'SUCCESS')
              Rows Removed by Filter: 12
        →  Index Scan using transactions_2026_03_account_date_idx on transactions_2026_03
              Index Cond: (account_id = 'ACC-001' AND created_at ≥ '2026-02-01' AND created_at ≤ '2026-03-15')
              Filter: (category = ANY ('{UPI,NEFT}') AND status = 'SUCCESS')
              Rows Removed by Filter: 8
Planning Time: 0.5ms
Execution Time: 12.8ms
```

---

8. Dynamic Filter Options (Facets)

8.1 The Problem

When a user selects `category=UPI`, the status filter should show counts reflecting the filtered data — e.g., "Success (32), Pending (3), Failed (1)" — not the totals across all categories. These are called **faceted counts**.

8.2 Implementation Strategy

Faceted counts are returned alongside the main query results. For the PostgreSQL path, we run a single query with `COUNT` + `GROUP BY`:

```sql
SELECT category, status, COUNT(*) as cnt
FROM transactions
WHERE account_id IN ('ACC-001')
  AND created_at BETWEEN '2026-01-01' AND '2026-03-31'
  -- note: category and status filters NOT applied here (for facet counts)
GROUP BY category, status;
```

This returns all category-status combinations, which the BFF assembles into facet counts:

```json
{
  "facets": {
    "categories": {
      "UPI": 45,
      "NEFT": 12,
      "RTGS": 3,
      "EMI": 8,
      "BILL_PAY": 5,
      "ATM": 15
    },
    "statuses": {
      "SUCCESS": 82,
      "PENDING": 4,
      "FAILED": 2
    },
    "types": {
      "CREDIT": 35,
      "DEBIT": 53
    }
  }
}
```

For the ES path, facets come from ES `aggregations` for free as part of the same search query.

8.3 Performance Consideration

The facet query adds ~5-10ms overhead. To avoid running it on every filter change (which is debounced at 300ms already), we cache facet counts for 30 seconds. The facet query runs with the "base filters" (account + date range) but without the "refinement filters" (category, status, type) so users can see how many results they'd get by toggling a refinement filter.

---

9. URL State Synchronization

9.1 Why URL Sync Matters

- **Bookmark:** User saves a filtered view as a browser bookmark.
- **Share:** User copies the URL to share a filtered view with a support agent.
- **Back/Forward:** Browser navigation traverses filter state history.
- **Deep link:** External links (email notifications) can open specific filtered views.

9.2 URL Schema

```
/transactions?accounts=ACC-001,ACC-002&from=2026-01-01&to=2026-03-31&cat=UPI,NEFT&minAmt=1000&maxAmt=50000&status=SUCCESS&type=DEBIT&page=
```

9.3 Implementation

```
function useSyncFiltersToURL(
  filters: TransactionFilters,
  dispatch: React.Dispatch<FilterAction>
) {
  const [searchParams, setSearchParams] = useSearchParams();

  // On mount: hydrate filters from URL
  useEffect(() => {
    const urlFilters = decodeFiltersFromURL(searchParams);
    if (Object.keys(urlFilters).length > 0) {
      dispatch({ type: 'HYDRATE_FROM_URL', payload: urlFilters });
    }
  }, []);

  // On filter change: update URL (debounced to avoid history spam)
  const debouncedSync = useDebounce(() => {
    const params = encodeFiltersToURL(filters);
    setSearchParams(params, { replace: true });
  }, 500);

  useEffect(() => { debouncedSync(); }, [filters]);
}

function encodeFiltersToURL(f: TransactionFilters): URLSearchParams {
  const params = new URLSearchParams();
  if (f.accountIds.length > 0)    params.set('accounts', f.accountIds.join(','));
  if (f.dateRange)                { params.set('from', f.dateRange.from); params.set('to', f.dateRange.to); }
  if (f.categories.length > 0)    params.set('cat', f.categories.join(','));
  if (f.amountRange?.min ≠ null) params.set('minAmt', String(f.amountRange.min));
  if (f.amountRange?.max ≠ null) params.set('maxAmt', String(f.amountRange.max));
  if (f.status ≠ 'ALL')           params.set('status', f.status);
  if (f.type ≠ 'ALL')             params.set('type', f.type);
  if (f.page > 0)                 params.set('page', String(f.page));
  if (f.pageSize ≠ 20)            params.set('size', String(f.pageSize));
  if (f.sortBy ≠ 'createdAt' || f.sortDir ≠ 'desc')
    params.set('sort', `${f.sortBy},${f.sortDir}`);
  return params;
}
```

`replace: true`: Uses `history.replaceState` instead of `pushState` during rapid filter changes. This prevents polluting the browser history with 50 entries for adjusting an amount slider. Only the final filter state gets pushed.

10. Pagination & Sorting Strategy

10.1 Cursor-Based vs. Offset-Based Pagination

Approach	Pros	Cons	Used When
Offset-based (`LIMIT 20 OFFSET 40`)	Simple, supports jumping to page N	Slow on deep pages (OFFSET scans discarded rows). Inconsistent if data changes between pages.	Default for transaction lists (users rarely go past page 5).
Cursor-based (`WHERE created_at < :cursor LIMIT 20`)	Consistent, fast on deep pages	Can't jump to page N. More complex API.	Infinite scroll on mobile (BFF-Mobile), export jobs.

10.2 Deep Pagination Guard

```
public Page<TransactionDTO> list(TransactionFilterDTO filters, Pageable pageable) {
    // Guard against deep pagination abuse
    if (pageable.getOffset() > 10_000) {
        throw new BadRequestException(
            "Maximum offset is 10,000. Use date range filters to narrow results, " +
            "or use the /export endpoint for bulk data."
        );
    }

    Specification<Transaction> spec = TransactionSpecBuilder.build(filters, userId);
    return transactionRepository.findAll(spec, pageable).map(mapper::toDTO);
}
```

10.3 Sort Options

Sort Field	SQL	Default?
`createdAt DESC`	`ORDER BY created_at DESC`	Yes — most recent first
`createdAt ASC`	`ORDER BY created_at ASC`	No
`amount DESC`	`ORDER BY amount DESC`	No
`amount ASC`	`ORDER BY amount ASC`	No

Sort by `relevance` is only available when a text search query (`q`) is present and routed to Elasticsearch.

---

11. Performance Engineering

11.1 Filter Response Time Targets

Scenario	Target (p99)	Achieved By
Single account, last 30 days, no other filters	< 50ms	Index scan on `(account_id, created_at)`, partition pruning
Multi-account, 90-day range, category + status	< 100ms	Composite index + filter pushdown
All accounts, 1-year range, all categories	< 300ms	Partition pruning limits to 12 partitions max
Filter with text search (`q` present)	< 200ms	Routed to ES (see Search Module doc)

11.2 Count Query Optimization

`SELECT COUNT(*)` for total results can be expensive on large datasets. Strategy:

1. **Exact count for small results (< 10K):** Run `COUNT(*)` — fast enough.
2. **Estimated count for large results:** Use `EXPLAIN` row estimate when count would be > 10K. Display as "~12,000 transactions" instead of "12,347 transactions."
3. **Pre-computed counts:** For the default view (last 30 days, all accounts), the count is cached in Redis with a 60-second TTL, invalidated on new transaction events.

```java
public long countTransactions(Specification<Transaction> spec) {
    long estimate = getExplainEstimate(spec);
    if (estimate < 10_000) {
        return transactionRepository.count(spec);  // exact count
    }
    return estimate;  // approximate count with ~ prefix in response
}
```

11.3 Connection Pool Sizing

Filter queries are read-only and can use PostgreSQL **read replicas** via a separate `DataSource`:

```yaml
spring:
  datasource:
    primary:
      url: jdbc:postgresql://pg-primary:5432/txn_db
      maximum-pool-size: 20
    replica:
      url: jdbc:postgresql://pg-replica:5432/txn_db
      maximum-pool-size: 30   # more connections for read-heavy filter workload
```

```java
@Transactional(readOnly = true)  // routes to read replica
public Page<TransactionDTO> list(TransactionFilterDTO filters, Pageable pageable) {
    // ...
}
```

---

12. Edge Cases & Error Handling

12.1 Invalid Filter Combinations

Edge Case	Handling
`from` > `to` (inverted date range)	400 Bad Request with clear error message
`minAmount` > `maxAmount`	400 Bad Request
Empty `accountIds` for non-admin user	Default to all user's accounts (not an error)
Unknown category value	400 Bad Request, list valid values in error detail
Date range > 3 years	400 Bad Request: "Maximum date range is 3 years. Use export for older data."
Page beyond result count	Return empty `results` array with `totalElements` (not an error)

12.2 Filter Validation (Backend)

```java
@Validated
public class TransactionFilterDTO {

    @Size(max = 10, message = "Maximum 10 accounts per query")
    private List<String> accountIds;

    @PastOrPresent(message = "Start date cannot be in the future")
    private Instant from;

    @PastOrPresent(message = "End date cannot be in the future")
    private Instant to;

    @Size(max = 7, message = "Maximum 7 categories")
    private List<@Pattern(regexp = "UPI|NEFT|RTGS|IMPS|BILL_PAY|EMI|ATM") String> categories;

    @DecimalMin(value = "0", message = "Minimum amount must be non-negative")
    private BigDecimal minAmount;

    @DecimalMax(value = "999999999.99", message = "Maximum amount exceeded")
    private BigDecimal maxAmount;

    @Pattern(regexp = "ALL|SUCCESS|PENDING|FAILED")
    private String status;

    @Pattern(regexp = "ALL|CREDIT|DEBIT")
    private String type;
}
```

12.3 Mobile-Specific Adjustments (BFF-Mobile)

Adjustment	Reason
Default page size = 10 (vs 20 on web)	Smaller viewport, less data per screen
Cursor-based pagination (infinite scroll)	Mobile UX convention
Compressed response (gzip)	Bandwidth savings on cellular
Fewer facet categories returned	Simplified filter UI on mobile
Date range presets only (no custom date picker)	Simpler mobile UX

---

13. GraphQL vs REST for Filters — Decision Rationale

13.1 The Argument for GraphQL in Filtering

Filtering is the one area where GraphQL's flexibility is most tempting. Different clients may want different response shapes:

```gql
# Mobile: lightweight, fewer fields
query {
  transactions(filters: { categories: [UPI], status: SUCCESS }) {
    results { id amount description createdAt }
    pagination { totalElements }
  }
}

# Web: full payload with facets and aggregations
query {
  transactions(filters: { categories: [UPI], status: SUCCESS }) {
    results { id amount description category status type createdAt referenceId }
    pagination { totalElements totalPages }
    facets {
      categories { key count }
      statuses { key count }
    }
  }
}
```

13.2 Why REST Still Wins for Our Filter Use Case

Factor	REST Advantage
Filter URL state sync	REST query params (`?cat=UPI&status=SUCCESS&from=2026-01-01`) map directly to browser URL. Bookmarkable, shareable, browser-back compatible. GraphQL queries are POST bodies — URL sync requires custom serialization of the GraphQL variables into URL params, adding unnecessary complexity.
HTTP caching	`GET /transactions?cat=UPI&status=SUCCESS` is cacheable at CDN, API Gateway, and browser level. GraphQL mutations and POST-based queries bypass HTTP caching entirely.
Pagination with `Link` headers	REST pagination uses standard `Link` headers, `X-Total-Count`, and URL-based page params. GraphQL pagination (cursor-based relay spec) is more complex and framework-specific.
Response shape is predictable	Filter results always have the same structure: `{ results[], pagination, facets }`. We never need 10 different response shapes for the same query. GraphQL's field selection flexibility is wasted here.
Backend query plan optimization	With REST, the backend knows the exact response shape at compile time. With GraphQL, the response shape is dynamic — the backend must resolve fields conditionally, which complicates query optimization and caching at the JPA/Hibernate level.

13.3 How BFF Already Solves the "Different Clients" Problem

The architecture uses separate BFF services rather than GraphQL to tailor responses:

```
┌──────────────────────────────────────────────────────────────┐
│  Instead of GraphQL field selection:                          │
│                                                               │
│  BFF-Web:    Returns full payload (20 fields, facets, pagination)
│  BFF-Mobile: Returns compact payload (8 fields, no facets,    │
│              cursor pagination, gzip compressed)              │
│                                                               │
│  Both call the same Transaction Service gRPC endpoint.         │
│  Each BFF maps the gRPC response to its client's needs.       │
│                                                               │
│  Cost: Two thin BFF services (< 500 lines each).              │
│  Benefit: Full HTTP caching, simple URLs, no GraphQL overhead. │
└──────────────────────────────────────────────────────────────┘
```

A full discussion of GraphQL trade-offs (security, caching, observability, rate limiting) is in the **Search Module doc, Section 13**.

End of Document — Filter Module v1.1